EXECUTIVE ACTION PLAN & ROADMAP

แผนปฏิบัติการเร่งด่วนเพื่อการขยายทีมพัฒนา (Executive Action Checklist)

การวางรากฐานทางวิศวกรรมซอฟต์แวร์ การจัดการความเสี่ยง การป้องกัน Git Merge Conflict และขั้นตอนการดำเนินงานสู่ทีม Multi-Developer

ชื่อโครงการ	**AI Agent Visual Workflow Builder (LGv3)**
ระดับความสำคัญ	**เร่งด่วนสูงสุด (Mission-Critical)**
สภาพแวดล้อมระบบหลัก	**https://aibuilder-dev.addpay.in.th**
เป้าหมายกรอบเวลา	**Roadmap 4 สัปดาห์ (พฤศจิกายน 2026)**
ผู้อนุมัติแผนงาน	**Executive / Project Sponsor**
วันที่จัดทำ	**30 ตุลาคม 2026**

1. วัตถุประสงค์และหลักการสำคัญ (Executive Objectives)

การขยายทีมพัฒนาจากผู้พัฒนาคนเดียว (Solo Dev) สู่การทำงานร่วมกันเป็นทีม (Multi-Developer Team) มีความท้าทายหลัก 3 ประการที่ต้องแก้ไขอย่างเร่งด่วน เพื่อป้องกันความสูญเสียจากโค้ดชนกันหรือระบบหลักขัดข้อง:

ACT-01	ACT-02	ACT-03	ACT-05
สลายโมดูล App.tsx	Git Branch Protection	Automated CI (GitHub)	แยก Staging Server

1. **ป้องกันปัญหาการชนกันของโค้ด (Zero Merge Conflicts):** โครงสร้างไฟล์ Frontend ปัจจุบันยังรวมศูนย์อยู่ที่ `App.tsx` หากมีผู้พัฒนามากกว่า 1 คนแก้ไขพร้อมกัน จะเกิดปัญหาการรวมโค้ดที่รุนแรง ต้องแยกชิ้นส่วนออกเป็นโมดูลย่อยโดยเร็ว
2. **รักษาเสถียรภาพของระบบบน Production:** ทุกการเปลี่ยนแปลงต้องผ่านการทดสอบอัตโนมัติ (Automated CI) และการตรวจสอบโค้ด (Peer Review) ก่อนขึ้นระบบจริงเสมอ ห้าม Push ตรงเข้าสู่ Branch `main`
3. **ลดเวลาเริ่มต้นทำงานของทีมงานใหม่ (Onboarding Time):** นักพัฒนาใหม่ทุกคนต้องสามารถ Clone โค้ดและรันระบบบนเครื่องตัวเองได้สำเร็จภายในเวลาไม่เกิน 15 นาที ผ่านคำสั่งที่เป็นมาตรฐาน

2. ตารางแผนปฏิบัติการเร่งด่วน (Urgent Action Matrix)

รหัสงาน	รายการงานที่ต้องปฏิบัติ (Action Item)	ผลกระทบหากไม่ทำ (Risk Impact)	ระยะเวลา	ความเร่งด่วน
ACT-01	**Refactor สลายไฟล์ `App.tsx` ออกเป็นโมดูลย่อย**	เกิด Git Conflict รุนแรงทันทีที่มี Dev เข้ามามากกว่า 1 คน	3 - 5 วัน	เร่งด่วนสูงสุด (P0)
ACT-02	**กำหนดนโยบาย Git Branching & Protection Rules**	เสี่ยงที่โค้ดบักหรือโค้ดที่ยังไม่พร้อมจะถูก Push ขึ้น Production ตรงๆ	1 วัน	เร่งด่วนสูงสุด (P0)
ACT-03	**ติดตั้งระบบ Automated CI (GitHub Actions)**	ต้องคอยรันเทสด้วยมือ เสี่ยงบักหลุดรอดเข้าสู่ระบบหลัก	1 - 2 วัน	สูง (P1)
ACT-04	**จัดเตรียม Docker Development Environment**	Dev ใหม่เสียเวลาติดตั้ง Dependency นาน และสภาพแวดล้อมไม่ตรงกัน	1 - 2 วัน	สูง (P1)
ACT-05	**แยกเซิร์ฟเวอร์ Staging ออกจาก Production บน Coolify**	การทดสอบฟังก์ชันใหม่จะรบกวนการใช้งานของผู้ใช้จริง	1 วัน	สูง (P1)
ACT-06	**วางโครงสร้าง Database Migration ด้วย Alembic**	เกิดข้อผิดพลาดของ Schema เมื่อฐานข้อมูลมีความซับซ้อนขึ้น	2 - 3 วัน	ปานกลาง (P2)

3. รายละเอียดขั้นตอนการลงมือปฏิบัติ (Action Item Deep-Dive)

🔴 ACT-01: Refactor สลายไฟล์ `App.tsx` ออกเป็นโมดูลย่อย (Critical)

> **สถานะปัจจุบัน:** ไฟล์ `frontend/src/App.tsx` มีขนาดถึง 27,000+ บรรทัด รวบรวมการทำงานแทบทุกอย่างไว้ด้วยกัน ซึ่งเป็นคอขวดที่อันตรายที่สุดเมื่อทำงานเป็นทีม

- สร้างโฟลเดอร์ `frontend/src/components/canvas/` : แยก `CanvasWorkspace.tsx` , `CanvasHeader.tsx` , `CanvasControls.tsx`
- สร้างโฟลเดอร์ `frontend/src/components/sidebars/` : แยก `LeftToolRail.tsx` , `NodePaletteDrawer.tsx`
- สร้างโฟลเดอร์ `frontend/src/components/panels/` : แยก `PropertiesInspector.tsx` , `ExecutionLogViewer.tsx`
- แยก Logic และ State ไปไว้ใน Custom Hooks: `useFlowNodes.ts` , `useWorkflowSync.ts`
- **เกณฑ์ความสำเร็จ:** ขนาดไฟล์ `App.tsx` ลดลงเหลือต่ำกว่า 500 บรรทัด ทำหน้าที่เป็น Shell ประกอบ Component เท่านั้น

🔴 ACT-02: กำหนดนโยบาย Git Branching & Protection Rules (Critical)

- กำหนดบทบาทของแต่ละ Branch ชัดเจน:
  - `main` : โค้ดที่พร้อมขึ้น Production (Protected)
  - `develop` : สาขาหลักสำหรับการรวมโค้ดและส่งทดสอบบน Staging (Protected)
  - `feat/<name>` : สาขาสำหรับฟังก์ชันใหม่ แตกออกจาก `develop`
  - `fix/<name>` : สาขาสำหรับแก้ไขข้อผิดพลาด
- เปิดใช้งาน **GitHub Branch Protection Rules**:
  - บังคับเปิด Pull Request (Require PR before merging)
  - บังคับมี Approvals จาก Lead Developer อย่างน้อย 1 ท่าน
  - บังคับให้ Status Checks (Automated Tests) ผ่านครบทุกข้อก่อน Merge

🟡 ACT-03: ติดตั้งระบบ Automated CI Pipeline (GitHub Actions)

- สร้างไฟล์คอนฟิก `.github/workflows/ci.yml` ทำงานอัตโนมัติทุกครั้งที่มีการเปิด Pull Request:
  1. **Backend Quality Gate:** ติดตั้ง Python 3.11 และรันคำสั่ง `uv run pytest` (ต้องผ่านครบ 24/24 Test Suites)
  2. **Frontend Quality Gate:** ติดตั้ง Node.js 20, รัน `npx tsc -b` (Type Check) และรัน `npm run build`
- **เกณฑ์ความสำเร็จ:** ป้องกันไม่ให้โค้ดที่มีข้อผิดพลาดถูก Merge เข้าสู่ระบบหลักโดยอัตโนมัติ

🟡 ACT-04: จัดเตรียม Docker Development Environment & Secrets

- สร้างไฟล์ `docker-compose.dev.yml` รองรับการ Mount Volume โค้ดเพื่อการแก้ไขแบบ Hot-Reload
- จัดทำคู่มือและไฟล์ `.env.example` ทั้งสองฝั่ง เพื่อให้ตั้งค่าได้ทันทีโดยไม่ต้องถามหา Key

🟡 ACT-05: แยกสภาพแวดล้อม Staging บน Coolify

- สร้าง Application ใหม่ในโปรเจกต์ `ai-builder` ชื่อ `ai-builder-staging`
- ผูกโดเมนย่อย: `https://aibuilder-staging.addpay.in.th` ดึงโค้ดจาก Branch `develop` อัตโนมัติ
- ช่วยให้ทีมสามารถทดสอบฟีเจอร์ใหม่ได้อย่างสบายใจ โดยไม่กระทบต่อผู้ใช้งานจริงบนระบบ Production

4. การประเมินความเสี่ยงและมาตรการป้องกัน (Risk Mitigation Matrix)

ความเสี่ยง (Risk Event)	โอกาสเกิด	ผลกระทบ	มาตรการป้องกันเชิงรุก (Mitigation Strategy)
Merge Conflict ใน `App.tsx`	**สูงมาก (95%)**	**วิกฤต**	เร่งปฏิบัติการ ACT-01 สลายโมดูลให้เสร็จสิ้นก่อนเปิดรับ Developer ใหม่
บั๊กหรือข้อผิดพลาดหลุดขึ้น Production	**ปานกลาง (50%)**	**สูง**	ติดตั้ง Automated CI Quality Gate (ACT-03) และแยกเซิร์ฟเวอร์ Staging (ACT-05)
API Keys หรือรหัสผ่านรั่วไหลลง Git	**ปานกลาง (40%)**	**สูง**	ใช้ไฟล์ `.env.example` และติดตั้ง Secret Scanning ใน CI Pipeline
ข้อมูลฐานข้อมูล Production เสียหายขณะปรับปรุง	**ต่ำ (20%)**	**วิกฤต**	ใช้ระบบ Auto-Migration ร่วมกับ Alembic และมีระบบ Backup Volume ประจำวันบน Coolify

5. แผนภูมิระยะเวลาดำเนินการ 4 สัปดาห์ (4-Week Implementation Roadmap)

ช่วงเวลา (Phase)	รายการงานที่ต้องทำให้เสร็จ (Key Deliverables)	ผลลัพธ์ที่เป็นรูปธรรม (Expected Milestone)
สัปดาห์ที่ 1 วางรากฐานระบบ	• กำหนด Git Branching Rules & Protection • ติดตั้ง GitHub Actions Automated CI Pipeline • จัดทำ `docker-compose.dev.yml` & `.env.example`	Repository มีระบบป้องกันโค้ด และมี CI คอยรันเทสอัตโนมัติ 100%
สัปดาห์ที่ 2 ปรับสถาปัตยกรรมโค้ด	• Refactor สลายไฟล์ `App.tsx` เป็น Component ย่อย • วางโครงสร้าง Alembic Database Migration	โค้ด Frontend แยกเป็นสัดส่วน พร้อมให้ Dev หลายคนแก้ไขพร้อมกันโดยไม่ชนกัน
สัปดาห์ที่ 3 เตรียมสิ่งแวดล้อม	• ตั้งค่าแอปพลิเคชัน Staging บน Coolify • เชิญ Developer กลุ่มแรกเข้ามาทดลอง Onboarding	มี Staging URL ใช้งานคู่ขนาน และ Dev ใหม่สามารถเริ่มงานได้ภายใน 15 นาที
สัปดาห์ที่ 4 เริ่มทำงานเป็นทีม	• เปิด Sprint แรก แบ่งงานผ่าน Task Board • ปฏิบัติตามมาตรฐาน Code Review และ Deploy รอบ Staging	ทีมพัฒนาหลายคนทำงานขนานกันได้อย่างมีเสถียรภาพและมีประสิทธิภาพสูงสุด

6. ดัชนีชี้วัดความสำเร็จ (Key Performance Indicators - KPIs)

1. **Zero Conflict Rate on Main:** ไม่เกิดความขัดแย้งของโค้ดบนสาขา `main` จากการวางกรอบงานที่ชัดเจน
2. **100% Test Pass Guarantee:** ทุก Pull Request ต้องผ่านการรันชุดทดสอบ 24/24 Tests สีเขียวทั้งหมด
3. **Developer Setup Time < 15 Mins:** นักพัฒนาใหม่สามารถ Clone และรันระบบพร้อมทำงานได้ภายใน 15 นาที
4. **Zero Production Downtime:** การอัปเดตระบบขึ้นสู่ Production ไม่มีผลกระทบต่อความพร้อมในการให้บริการ